Technical Architecture Document: UpTimeEngine

Single-page architecture analysis based on the repository structure, runtime entry points, and core integration patterns.

1. System Overview & Paradigms

Core Paradigm: UpTimeEngine is a layered, agentic investigation platform rather than a classic CRUD application. Its central pattern is a runtime that builds a prompt, loads a dynamic tool catalog, invokes an LLM, executes tool calls against live infrastructure data, captures results, and iterates until it can deliver an operational conclusion.

Architectural style: hybrid of layered architecture and tool-driven agent orchestration. The repository separates configuration, orchestration, tool execution, source integrations, and rendering surfaces.

Layer	Concrete technology	Architectural role
Presentation / CLI	Typer, Rich	User entry point for ask, investigate, checks, and tool management
API / server surface	FastAPI, Uvicorn	REST endpoints for admin and health-check execution
Configuration	Pydantic models, YAML, env overrides	Combined model for runtime, credentials, toolset selection, and model registry
Model access	LiteLLM and provider abstraction	Normalizes LLM access across providers and model metadata

Layer	Concrete technology	Architectural role
Agent orchestration	ToolCallingLLM, ToolExecutor, prompt generation	Controls iterative investigation and tool usage
Tool integration	Python toolsets, MCP servers, kubectl/bash wrappers, HTTP integrations	Connects to Prometheus, Kubernetes, cloud APIs, and SaaS systems
Data services	Supabase DAL, file caches, config/status JSON	Stores metadata, OAuth state, and operational status
Telemetry / observability	Sentry, OTel, Braintrust, structured logging	Captures runtime error and usage data for operations and experiments
Infrastructure integration	Kubernetes, AlertManager, Jira, PagerDuty, OpsGenie, Slack, GitHub, databases	External systems queried or updated during investigations

System boundaries

- **Handled directly by the repository:** CLI startup, server setup, configuration, LLM orchestration, toolset lifecycle, prompt generation, plugin connections, admin endpoints, and health-check execution.
- **Delegated to external infrastructure:** model providers, Kubernetes clusters, observability backends, ticketing systems, cloud APIs, and telemetry services.

2. Directory Taxonomy & Component Mapping

- **UpTimeEngine/:** main application runtime; contains the CLI, configuration, orchestration framework, and shared utilities.
- **UpTimeEngine/core/:** orchestration and domain execution heart of the platform. Contains the LLM loop, toolset manager, prompt logic, tracing, usage recording, and

scheduled operations.

- **UpTimeEngine/plugins/:** pluggable extension layer responsible for sources, destinations, toolsets, prompts, and skills.
- **UpTimeEngine/admin/:** administrative FastAPI surface for reloading toolsets and models at runtime.
- **UpTimeEngine/checks/:** health-check execution framework and API entry points.
- **UpTimeEngine/utils/:** cross-cutting helpers for auth, env setup, file handling, logging, formatting, and runtime support.
- **server.py:** API server bootstrap and long-lived runtime wiring; initializes config, tracing, status sync, admin and checks routes.
- **tests/:** functional, integration, and LLM evaluation coverage for regressions and platform behavior.
- **docs/:** user documentation and site content used for installation, platform docs, walkthroughs, and toolset reference pages.
- **helm/:** deployment packaging and Kubernetes chart resources.
- **scripts/:** supporting operational workflows such as sandbox setup, benchmarking, and environment prep.

Key concrete modules

- `UpTimeEngine/main.py`: public CLI entry point; wires prompts, toolsets, and investigation commands.
- `UpTimeEngine/config.py`: config model, environment fallback logic, and tool/LLM creation factories.
- `UpTimeEngine/core/tool_calling_llm.py`: iterative tool-call loop, approval behavior, and result handling.
- `UpTimeEngine/core/toolset_manager.py`: loads built-in/custom toolsets and executes prerequisite checks.
- `UpTimeEngine/core/llm.py`: model registry and provider configuration.
- `UpTimeEngine/plugins/toolsets/__init__.py`: built-in toolset composition including Prometheus, Kubernetes, Grafana, Docker, database, and MCP integrations.
- `UpTimeEngine/plugins/interfaces.py`: source and destination plugin contracts.
- `UpTimeEngine/plugins/sources/prometheus/plugin.py`: AlertManager issue ingestion logic.

- `UpTimeEngine/admin/admin_api.py`: admin API to reload models and toolsets live.
- `UpTimeEngine/checks/checks_api.py`: health-check execution endpoint for operator-style checks.

3. End-to-End Data Flow & Lifecycle

1. **Ingress / transport:** A user invokes an ask or investigate command via Typer in `UpTimeEngine/main.py`, or a server API route in `server.py` initiates a request.
2. **Validation / configuration:** `Config.load_from_file()` or `Config.load_from_env()` merges YAML and environment input, resolves model settings, toolset config, and credentials.
3. **Toolset bootstrap:** the ToolsetManager loads built-in and custom toolsets, checks prerequisites, filters by tag, and prepares a runnable ToolExecutor.
4. **Prompt construction:** `build_chat_messages()` and prompt-building helpers create the system and user content based on the current tool inventory and skill catalog.
5. **Application / orchestration:** `ToolCallingLLM` sends messages to the LLM and handles the iterative tool-call cycle, approval logic, and context management.
6. **Domain / business logic:** the model decides which external systems and tools to query, aligned with the available toolset schema and operational instructions.
7. **Persistence / egress:** tool outputs are returned as structured tool results, sent through the user-facing formatter or destination plugin, and optionally recorded to telemetry or ticket sources.

The runtime design is intentionally data-flow-heavy and live-query-based: it retrieves operational evidence from external systems instead of maintaining a local warehouse as the primary source of truth.

4. State Management & Data Architecture

Data models & boundaries

The repository uses a clear but pragmatic boundary model:

- **Config models:** define environment-driven state such as model selection, API keys, and source credentials in `UpTimeEngine/config.py`.
- **Tool metadata:** parameter schemas and execution contracts live in `UpTimeEngine/core/tools.py`.
- **Issue models:** incident data from Prometheus, Jira, PagerDuty, and related sources are normalized into internal issue representations.
- **Runtime payloads:** chat messages, tool results, and LLM output are passed as structured objects rather than being stored in a monolithic domain model.

Persistence strategy

- Static config is persisted as YAML and environment configuration.
- Toolset readiness is cached as status JSON and refreshable at runtime.
- Supabase acts as a metadata and identity layer for runtime state, OAuth, and skill hierarchy.
- Tool outputs are often stored to disk or streamed for memory-safe processing rather than all being retained in memory.

Event / messaging system

The project does not implement a centralized event bus like Kafka or RabbitMQ as the primary runtime control-plane. Instead, it relies on:

- background conversation worker flows under `UpTimeEngine/core/conversations_worker`,
- scheduled prompt execution under `UpTimeEngine/core/scheduled_prompts`,
- event callbacks from the interactive loop and tracing system,
- and operational notifications to Slack and other destinations.

This is effectively an event-driven orchestration model without forcing a heavy broker topology.

5. Cross-Cutting Concerns

Authentication & authorization

- Credentials are supplied through YAML config or environment variables and then merged into the Config model.
- API-layer auth and session helpers exist in `UpTimeEngine/utils/auth.py`.
- OAuth flows are supported via `UpTimeEngine/core/oauth_config.py`, `UpTimeEngine/core/oauth_utils.py`, and related callback management.
- The project explicitly positions itself as honoring read-only access and RBAC assumptions, which is important in production incident investigation.

Observability

- Structured logging is configured in `server.py` and utility layers.
- Sentry is enabled for runtime diagnostics and error reporting.
- OpenTelemetry and Braintrust tracing hooks are integrated for request and experiment telemetry.
- Usage accounting and token accounting are tracked in `UpTimeEngine/core/usage_recorder.py` and `UpTimeEngine/core/llm_usage.py`.

Error handling & resiliency

- Toolsets are health-checked independently; a failed integration does not necessarily crash the platform.
- Prerequisite checks are time-bounded to avoid blocked startup from a single stale datasource.
- Approval gates and safeguards reduce unsafe or repeated tool calls.
- Tool outputs are normalized into structured statuses such as success, error, no_data, approval_required, and frontend_pause.

Architecture Summary

UpTimeEngine is a modular AI operations runtime whose core abstraction is a dynamic toolset registry plus an iterative LLM execution loop. This design enables the platform to investigate live operational systems in a provider-flexible and tool-safe manner while preserving strong observability and graceful degradation under partial system failure.

In practical terms, the repository is built as a production-oriented incident engine: it gathers evidence from live infrastructure, normalizes it into a common investigation model, reasons over it with an LLM, and then emits a diagnosis or action path to human operators or downstream systems.